

Can Recurrent Neural Networks (RNNs) Be Used to Simulate Driver Mobility Data Effectively? An evaluation of **TravNet**.

March 2025

1 Introduction

Realistically simulated mobility data is highly desired in studies predicting electric vehicle (EV) charging demand. EV charging demand is frequently predicted using real charging data from pilot studies. However, the issue with this approach is that charging demand can be highly heterogeneous to the study involved. Mobility data, inspired by travel surveys, is intended to reflect the population as a whole. Under the assumption that EV owners and combustion engine vehicle owners (CEVs) exhibit the same travel patterns. Predicting EV demand is a popular research question as it is an issue of unprecedented importance. The adoption of EVs is growing rapidly and as individuals tend to charge vehicles at similar times (i.e. after work), this represents an unanticipated load on the grid. Understanding charging demand can help electricity providers adapt through upgrading distribution networks and/ or presenting demand-response schemes (Yaghoubi et al. 2024; Pareschi et al. 2020).

My project seeks to showcase **TravNet**, a highly-customised RNN-based neural network with matrix-based losses and rationality penalties, designed to generate weekly car travel schedules for individuals with different characteristics. My target variables of interest (at the trip level) are: **TripStart**, **Duration**, **Distance** and **Purpose**. **TravNet** will take the form of a Python package and will be made available on PyPi. It is intended to be adaptable to various use cases, well documented, suitable for further development and intuitive to use for those with a basic understanding of Python.

According to the literature and the best of my knowledge, the targets are obtained in one of two ways:

1. Building a data-driven simulation model from travel survey data. (Lojowska et al. 2012)
2. Assuming the target variables take probabilistic functions and drawing samples from those functions. (Liu, Tayarani, and Gao 2022).

Using Neural Networks would fall under category (1). Researchers usually opt for Monte Carlo, copulas, and Markov chain modeling (Lojowska et al. 2012; Wang and Infield 2018). To my knowledge, Jahangir et al. 2019 are the only researchers who have used neural networks. They also used an RNN and obtained performance metrics for key outcome variables and compared them to traditional methods. However, the objective of their study is to predict the next trip following previous trips rather than generate full travel schedules. It is worth mentioning that individual-level features are seldom used in the literature.

2 Data Set & Methodology

The UK National Travel Survey (NTS) will be used for this project (Transport 2023). This is a rich collection of datasets covering trip-level, individual-level, household-level, vehicle-level, region-level and day-level information. When fully merged the dataset contains travel data from 2002 until 2023, it spans 5,000,000+ rows and 400+ columns. The highest granularity is at the trip level. In addition to classical data-cleaning procedures such as removing missing values, further transformations are required to make the data suitable for an RNN, details of which are below.

2.1 Data Wrangling & Cleaning

- (1): Trip, day, individual, household, region and vehicle datasets were merged on their unique identifiers to obtain a trip-level dataset. Unless one has a lot of RAM, the **loader** function merges data year by year and uses **survey_year** as the parameter. Columns with more than 30% missing values are dropped - this parameter is user-defined.
- (2): Correlation plots of all variables in the dataset with **Purpose** and **Distance** were obtained. The union of all variables with a correlation greater than ρ (user-defined) can be saved by the user. For the development of the model, ρ was set to 0.2. The model variable selection process is relatively automated. However, once correlated variables are selected, the developer has to choose how they are encoded and if there is any multi-collinearity (i.e. the same variables but in different granularity). In addition to the targets, the features displayed in **appendix A** have been selected. Most were MinMax encoded, as the correlation plots treated all variables as numerical. Some of the temporal variables used cyclical encoding to reflect their seasonality ¹. **TripStart** and **TripEnd** used min-max scaling, from which **Duration** was obtained. **Distance** used logarithmic scaling as it was heavily skewed.

¹<https://medium.com/@axelazara6/why-we-need-encoding-cyclical-features-79ecc3531232>

- (3): Target variables were transformed to wide format. In wide format, these are grouped by the sequence they take for a given day of the week. Thus the 3rd trip on a Wednesday would have a 3 in the `DayOfWeek` column, and values in `TripStart_3`, `Duration_3` etc. There were outlier individuals who may have travelled 15+ times in a day. Therefore the maximum number of trips was reduced to 10 (to cover over 99% of the population and to reduce sparsity). This parameter can be easily changed by the developer. These transformations can be seen dynamically in `analysis.ipynb`.
- (4): Matrices representing weekly travel diaries were created for each individual. These were originally of shape (7, 78). 7 being the days of the week and 78 including the long-transformed target columns along with any non-target features. As the RNN is built to have a memory of 7 periods (1 week) a masking of 7 days was appended to the start of each individual-level matrix. A 7-row input matrix slides into the RNN and the predictions are fed back into the input layer to predict the next day's travel. RNNs expect inputs of the shape (*sequence_length*, *batch_size*, *input_features*). Therefore, for each individual their travel diary matrix was of shape (14, 1, 78). There were 6,775 individuals in the 2017 datasets and so my final input tensor was of shape (6775, 14, 1, 78).
- (5): We train, test and evaluate! The pseudo-code is available below and the actual code is in `TravNet.py`

2.2 Training Loop

Algorithm 1 Training Algorithm

```

for e in epoch do

    for Individual, i, with target and non-target features do

        for day in day of the week in travel diary do
            Use TravNet to forecast outcome variables for next day. Using a sequence length (memory) of 7 days.
            Feed predictions back into the input matrix.
            Use predictions for each day to create a predicted weekly travel matrix.
        end for

        Use predicted weekly travel matrix and the true travel matrix for that individual to calculate matrix-based losses.
        Apply any rewards/ penalties to the loss function to make predictions resemble real travel data.
        Update Weights & biases.
    end for

end for

```

2.3 Model Design

An RNN was chosen as the base architecture as it is designed for sequential data like mine. And, as a neural network it can pick up on complex interactions and non-linearities, which are to be expected for highly stochastic travel data. With PyTorch, it also has much opportunity for customization to fit my unconventional use case. One should turn to **appendix B** to see a summary of the architecture. The Adam optimiser was chosen as it is the generally accepted state of the art, and a learning rate was chosen through experimentation. One might ask why the input layer (`rnn.weight_ih_10`) has 308 weights, while the input tensor has only 78 features. This summarises one of the major learning curves for this project - computation graph differentiability. `CrossEntropyLoss` expects logits as outputs and so we reshape `output_purp` to (10,24), where 10 is the max number of trips for a day and 24 is 23 trip categories plus a no trip category of 0. Initially, I used `torch.argmax` on the logits and fed these back into the input matrix for the next day. However, `argmax` is not differentiable and so the model cannot learn from earlier incorrect `Purpose` predictions. Consequently, I converted the `output_purp` into a flattened size-240 softmaxes and fed them back for the next days inputs.

```

1 y_ts_pred_7, y_dur_pred_7, y_distance_pred_7, y_purpose_pred_7, y_istrip_pred_7, is_trip_mask = rnn_model
  .forward(input_matrix)
2
3 # INCORRECT
4 categorical_prediction7 = torch.argmax(y_purpose_pred_7, dim=-1)
5
6 # INSTEAD
7
8 # Apply temperature-scaled softmax
9 y_purpose_softmax = F.softmax(y_purpose_pred_7 / temperature, dim=-1)
10
11 # Reshape back to match your input matrix (flattened)
12 y_purpose_softmax = y_purpose_softmax.reshape(1, 1, max_journey_seq*NUM_CLASSES)
13
14 # Feed back for next days predictions
15 new_input_row = torch.concat([y_ts_pred_7, y_dur_pred_7, y_distance_pred_7, y_purpose_softmax,
  y_istrip_pred_7, X_features[seq_length+t,:].unsqueeze(1)], dim=2)

```

2.3.1 Specifics: Penalties, Rewards and Losses.

Initially, differentiable penalties were imposed to the total loss so that sequential rationality conditions such as $TripStart_t > TripStart_{t-1}$ were met. However, after enough training epochs and using **Duration** instead of **TripEnd** the model was able to learn these itself. An entropy reward was added to discourage a homogenous **Purpose** output.

```
1 # Calculating entropy across trip
2
3 entropy_t = -(y_purpose_softmax.reshape(max_journey_seq, NUM_CLASSES) * torch.log(y_purpose_softmax + 1e
4     -8)).reshape(max_journey_seq, NUM_CLASSES)).sum(dim=1)
5
6 entropies.append(entropy_t)
7
8 purpose_entropy = torch.stack(entropies, dim=0).mean()
9
10 total_loss = ... + entropy_weight*purpose_entropy
```

As most people travel only 2/3 times a day but the travel matrix allows up to 10 trips, 70%+ of trips were of **Purpose** = 0. Thus, I had a serious issue of mode collapse. To bypass this I set **ignore_index=0** for the Cross-entropy loss function.

This means that the model only calculates losses for when trips actually occurred. Additionally, the **Purpose** entries for the true travel week was used as a mask for the total final loss (across all targets) so that the model would only be judged on how it predicted trips that actually happened.

```
1 purpose_mask = torch.clamp(true_matrix_purpose.reshape(7,max_journey_seq),max=1)
2
3 ts_loss = mse_loss(prediction_matrix_ts, true_matrix_ts)
4 ts_loss = (ts_loss*purpose_mask).sum() / (purpose_mask.sum() + 1e-6)
5 ...
6 distance_loss = mse_loss(torch.log(1+prediction_matrix_distance), torch.log(1+true_matrix_distance) )
7 distance_loss = (distance_loss*purpose_mask).sum() / (purpose_mask.sum() + 1e-6)
```

This comes with the issue that the model will not be able to predict trips not happening. To bypass this I introduced another target, **IsTrip** that predicts the occurrence of a trip on a given slot on a given day. This also suffers from mode collapse. However, in my model's forward pass as well as outputting logits I outputted a bernoulli mask, which used the sigmoid probabilities of **IsTrip** to randomly sample the occurrence.

```
1 y_mask = torch.bernoulli(y_istrip_pred_sigmoid).unsqueeze(1)
```

This is non-differentiable. However, this was only used on top of the final output travel matrix and not to calculate the losses.

3 Results & Evaluation

We can instantly see from **Appendix C** that loss is volatile. This is to be expected as we are trying to predict human mobility, which is inherently highly stochastic. Nevertheless, we tend to see a reduction across all the losses, which stabilises after around 1000 individuals. Interestingly we see that after around 6000 individuals entropy begins to increase so it is possible that if more data was used we could see more varied trips. However, the true performance of **TravNet** has to be inferred on the aggregate level. Once mobility data has been generated from a sample of individuals, we need to create distributions of **TripStart**, **Duration**, **Distance** for different **TripPurpose** and compare the distributions to the real data.

The final product is **TravelNet.generate_travel_data(N)** found under **Modules.TravNetUser**, where N is the number of individuals that the user wants to generate weekly travel diaries for. To obtain all non-target features N individuals are randomly sampled from the original training set and are fed into the model which generates their travel diaries. This is not a conventional prediction task with a training and test set but more closely resembles a generative task, where the model is expected to generate travel diaries for a user with given characteristics. The original training data is used only to sample individual characteristics for evaluation purposes and the model cannot see the true target variables of these individuals. Hence, there is no issue of data leakage.

Using **generate_travel_data(N)**, we can begin to explore how the model performs on aggregate. First we look at how well it predicts the proportions of different trip purposes. There are 23 categories in total. However, the model only predicts 1 (61% of the time), 5 (17%), 10 (15%) and 23 (7%). They are distributed as follows in the real data: 20%, 11%, 10% and 6% respectively.

We can see that the model heavily overpredicts 1, fails to predict the rarer categories but for 5, 10 and 23 the model comes much closer to the true proportions. Next we look at aggregate statistics for all other target variables for purpose categories 1,5,10 and 23.

My model appears to capture the ordinal relationships for **TripNum** and **TripStart** quite well, in regards to the median. It overshoots **Duration** and doesn't capture any meaningful order for **Distance**. The most problematic however is that it fails to pick up on the strong variability present across travel data. This is even more apparent in the histograms below.

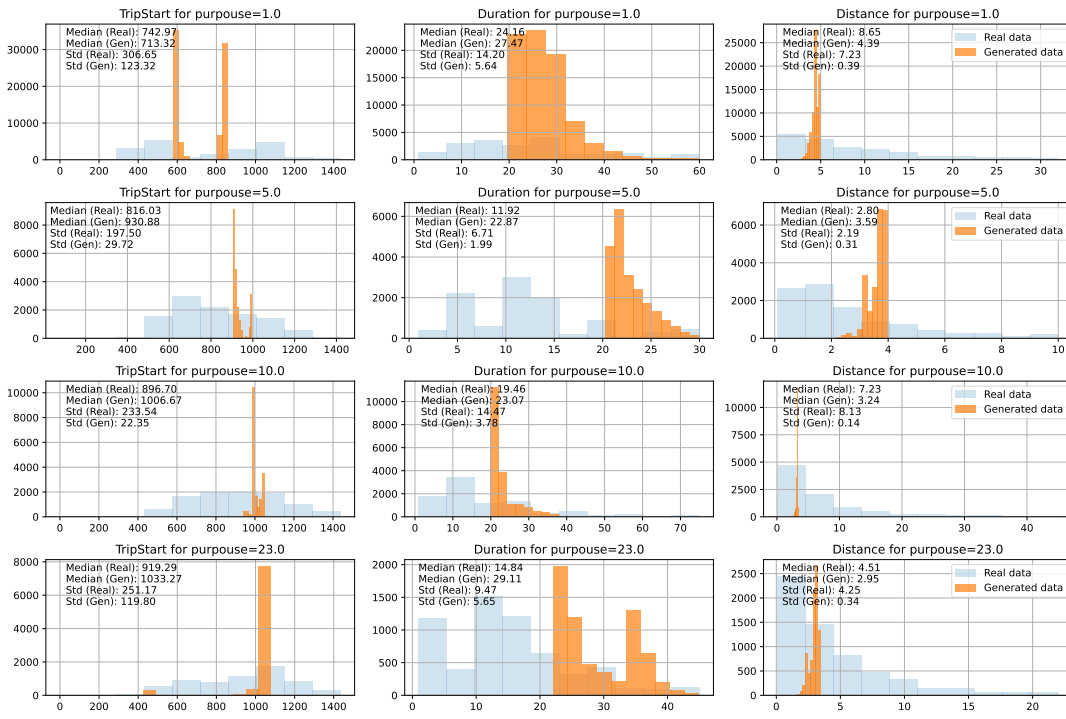


Figure 1: Histograms of true distributions and generated distributions, for predicted Purpose.

Although not formally tested, another item to mention is that **TravNet** is extremely fast at generating data. 10,000 individuals were generated in around 1m 30s and it appears to have $O(N)$ complexity. On a previous project, I tried to use copulas to generate travel data and it took 30 minutes to generate 1,000 individuals, on a cluster!

4 Further Research & Conclusion

The following were not carried out due to time constraints but are good areas for further study:

- **Variable selection**
 - **Purpose** and **Distance** were chosen for the correlation plots. However, all target variables could be considered.
 - Although, most variables are categorical and have an ordinal relationship, for those that do not, correlations should be plotted with their dummies. Due to time and resource constraints, this was infeasible but should be explored in future studies.
- **Model architecture**
 - A single RNN layer was trialled. However, one should experiment with multiple layers, perhaps intertwined with fully connected layers.
 - An LSTM was initially considered. However, the sequence length is short (7-days). An LSTM is better suited to longer-term dependencies.
- **More data**
 - Only data for 2017 was used, which is only a small proportion of the dataset, which ranges from 2002 - 2023. Due to computational constraints a fuller range of data could not be used.

This article describes **TravNet v1.0.0**, a user-friendly Python package that uses an RNN-based architecture and rich travel data to generate weekly car travel dairies for a user defined set of individuals. Although, the results are not too promising as of yet I believe this model has potential. The fact that it is fast and scalable is useful for this domain. The most important problem to solve now is to find a way to learn variability, as RNNs are designed for prediction tasks and thus generating a precise point estimate. Some approaches I have come across include using dropout neurons and learning noise parameters from the data. All-in-all, this has been a fascinating journey in understanding the intricacies of PyTorch, how to customise all parts of a model and the training loop to suit a use case unconventional to how RNNs are typically used. I hope to work on this project going forward and to have a solution to modelling variability in **v2.0.0**.

References

Jahangir, Hamidreza et al. (2019). "Charging demand of Plug-in Electric Vehicles: Forecasting travel behavior based on a novel Rough Artificial Neural Network approach". In: *Journal of Cleaner Production* 229, pp. 1029–1044. ISSN: 0959-6526. DOI: <https://doi.org/10.1016/j.jclepro.2019.04.345>. URL: <https://www.sciencedirect.com/science/article/pii/S0959652619314428>.

- Liu, Yuechen Sophia, Mohammad Tayarani, and H. Oliver Gao (2022). “An activity-based travel and charging behavior model for simulating battery electric vehicle charging demand”. In: *Energy* 258, p. 124938. ISSN: 0360-5442. DOI: <https://doi.org/10.1016/j.energy.2022.124938>. URL: <https://www.sciencedirect.com/science/article/pii/S0360544222018382>.
- Lojowska, Alicja et al. (Nov. 2012). “Stochastic Modeling of Power Demand Due to EVs Using Copula”. In: *Power Systems, IEEE Transactions on* 27, pp. 1960–1968. DOI: 10.1109/TPWRS.2012.2192139.
- Pareschi, Giacomo et al. (2020). “Are travel surveys a good basis for EV models? Validation of simulated charging profiles against empirical data”. In: *Applied Energy* 275, p. 115318. ISSN: 0306-2619. DOI: <https://doi.org/10.1016/j.apenergy.2020.115318>. URL: <https://www.sciencedirect.com/science/article/pii/S0306261920308308>.
- Transport, Department for (2023). *National Travel Survey*. UK Data Service. [data series]. 8th Release. DOI: 10.5255/UKDA-Series-2000037. URL: <http://doi.org/10.5255/UKDA-Series-2000037>.
- Wang, Yue and David Infield (Jan. 2018). “Markov Chain Monte Carlo simulation of electric vehicle use for network integration studies”. In: *International Journal of Electrical Power Energy Systems* 99, pp. 85–94. DOI: 10.1016/j.ijepes.2018.01.008.
- Yaghoubi, Elaheh et al. (2024). “A systematic review and meta-analysis of machine learning, deep learning, and ensemble learning approaches in predicting EV charging behaviour”. In: *Engineering Applications of Artificial Intelligence* 135, p. 108789. ISSN: 0952-1976. DOI: <https://doi.org/10.1016/j.engappai.2024.108789>. URL: <https://www.sciencedirect.com/science/article/pii/S0952197624009473>.

Appendices

A Post-variable Selection

Index	Variable	Description	Encoding/ Type
0	TripStart_1	Trip start time (minutes from midnight)	MinMax
1	TripEnd_1	Trip end time (minutes from midnight)	MinMax
2	TripStart_2	...	...
...	...	...	...
18	TripStart_10	...	...
19	TripEnd_10	...	...
20	TripDisExSW_1	Trip distance excluding short walk (miles)	logarithmic
21	TripDisExSW_2	...	...
...	...	...	...
29	TripDisExSW_10	...	...
30	TripPurpose_B01ID_1	Trip purpose,	23 categories
31	TripPurpose_B01ID_2	...	
...	...	...	
39	TripPurpose_B01ID_10	...	
40	IsTrip_1	If a trip happened for that slot of the day	Binary
...	...	...	...
49	IsTrip_10	...	...

Table 1: Target variables.

Index	Variable	Description
50	DrivLic_B02ID	Type of driving licence held - Summary banding - 3 categories
51	VehAnMileage	Vehicle annual mileage
52	HHoldEmploy_B01ID	Number of employed in household - broken down by full and part time workers
53	VehComMile_B01ID	Annual vehicle commuting mileage - banded mileage
54	EcoStat_B02ID	Working status of individual - Summary - 6 categories
55	HHoldNumPeople	Number of people in household
56	WkPlace_B01ID	Work place location
57	Age_B01ID	Age of person - banded age - Band A - All ages - 21 categories
58	HHoldStruct_B02ID	Household structure - 6 categories
59	HRPWorkStat_B02ID	Household Reference Person (HRP) - working status - 4 categories
60	HHoldNumChildren	Household number of children
61	EcoStat_B03ID	Working status of individual - Summary - 4 categories
62	EducN_B01ID	Currently attending school or college - full or part time
63	TravelYear	Year trip took place
64	TWSMonth_cos	Month trip took place (cyclical encoding)
65	TWSMonth_sin	...
66	TravelWeekDay_B01ID_cos	Weekday trip took place (cyclical encoding)
67	TravelWeekDay_B01ID_sin	...
68	PSUGOR_B02ID_0.0	PSU region (one-hot encoding)
79	PSUGOR_B02ID_1.0	...
70	PSUGOR_B02ID_2.0	...
...	...	...
77	PSUGOR_B02ID_9.0	...

Table 2: Non-target Variables

Note: Non-target variables used numerical MinMax encoding with Sklearn’s **StandardScaler** unless mentioned otherwise.

B Model Architecture

Layer / Parameter	Shape
Recurrent Layer	
Input weight (<code>rnn.weight_ih_l0</code>)	[9, 308]
Hidden weight (<code>rnn.weight_hh_l0</code>)	[9, 9]
Input bias (<code>rnn.bias_ih_l0</code>)	[9]
Hidden bias (<code>rnn.bias_hh_l0</code>)	[9]
Output Heads	
Trip start time (<code>output_ts</code>)	Weight: [10, 9]; Bias: [10]
Trip duration (<code>output_dur</code>)	Weight: [10, 9]; Bias: [10]
Trip distance (<code>output_dist</code>)	Weight: [10, 9]; Bias: [10]
Trip purpose (<code>output_purp</code>)	Weight: [240, 9]; Bias: [240]
Trip binary indicator (<code>output_bin</code>)	Weight: [10, 9]; Bias: [10]
Loss Functions & Config	
Categorical (purpose)	CrossEntropyLoss (weighted, <code>ignore_index = 0</code>)
Continuous (start time, duration, distance)	MSELoss (reduction=“none”)
Binary (istrip)	BCEWithLogitsLoss
Optimiser	Adam
Learning rate	0.0005

Table 3: TravNet Model Architecture Summary

C Training Losses

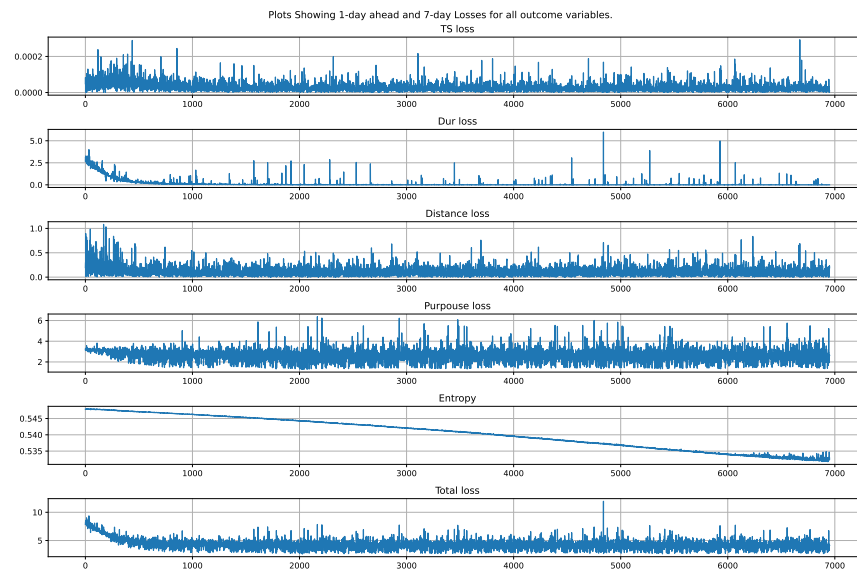


Figure 2: A plot of Individual and Total Losses.